تحليل وتصميم الخوارزميات

المحاضرة السادسة : خوارزميات الترتيب (الجزء الثاني) - الترتيب بالإضافة وترتيب شيل

في المحاضرة السابقة، درسنا خوارزميتي الترتيب بالاختيار والترتيب الفقاعي، ورأينا أن كفاءتهما في الحالة الأسوأ هي $O(n^2)$. اليوم، سنتعلم خوارزمية جديدة من نفس التعقيد، وهي "الترتيب بالإضافة"، لكنها تتميز بأداء استثنائي في بعض الحالات. بعد ذلك، سنرى كيف يمكن "تحسين" هذه الخوارزمية بشكل كبير لنحصل على أداء أسرع بكثير مع "ترتيب شيل."

1.الترتيب بالإضافة(Insertion Sort)

الفكرة الأساسية :هل سبق لك أن رتبت أوراق اللعب في يدك؟ على الأغلب، أنت تستخدم الترتيب بالإضافة بشكل طبيعي! الفكرة هي أنك تبني الجزء المرتب من القائمة تدريجياً، عنصراً تلو الآخر.

خطوات الخوارزمية (لترتيب تصاعدي):

الخطوات التالية هي الخطوات العامة لترتيب أي قائمة أو مجموعة من البيانات **تصاعديًا ((Ascending Order**، حيث تكون النتيجة النهائية هي أن العناصر الأصغر تأتي أولاً والأكبر تأتي لاحقًا (من الأصغر إلى الأكبر).

خطوات الترتيب التصاعدي ((General Sorting Steps

1. تحديد نقطة البداية (الافتراض المبدئي)

- **افترض** أن أول جزء من القائمة (أو المجموعة) قد تم ترتيبه بالفعل. في معظم الخوارزميات، يُعد العنصر الأول هو الجزء المرتب المبدئي.
- **ابدأ** عملية الترتيب الفعلي من العنصر **الثاني** في القائمة (الموقع ذو الفهرس 1).

2. اختيار المفتاح ((Key Selection

- **اختر** العنصر الحالي في الجزء غير المرتب واجعله **"المفتاح" ((Key** الذي سيتم إدراجه في الجزء المرتب.

3. مقارنة المفتاح بالجزء المرتب

- **قارن** المفتاح (العنصر الحالي) بجميع العناصر التي تسبقه (الجزء المرتب) بدءًا من العنصر الأخير في هذا الجزء.
- **الشرط التصاعدي:** يجب أن يكون الهدف هو إيجاد موضع يدخل فيه المفتاح بحيث يظل **أصغر** من العنصر الذي يليه في الجزء المرتب، و**أكبر** من العنصر الذي يسبقه.

4. الإزاحة للعناصر الأكبر ((Shifting

- **إذا كان** المفتاح أصغر من العنصر الذي تتم مقارنته به (في حالة الترتيب التصاعدي)، قم **بإزاحة** ذلك العنصر الأكبر خطوة واحدة إلى اليمين.
- **كرر** عملية الإزاحة والمقارنة مع جميع العناصر السابقة في الجزء المرتب حتى تجد عنصرًا أصغر من المفتاح أو تصل إلى بداية القائمة.

5. إدراج المفتاح ((Insertion

- **أدرج** المفتاح (العنصر الأصلي) في الموضع الفارغ الذي نتج عن الإزاحات (أي في الموضع الذي تكون فيه جميع العناصر على يساره أصغر منه، وجميع العناصر على يمينه أكبر منه).

6. التكرار والإنهاء

- **انتقل** إلى العنصر التالي في القائمة غير المرتبة (ارجع إلى الخطوة 2).
- **كرر** العملية حتى يتم معالجة جميع العناصر وإدراجها في الجزء المرتب.
- عند الوصول إلى نهاية القائمة، تكون القائمة بأكملها مرتبة **تصاعديًا** (من الأصغر إلى الأكبر).

يُعد **الترتيب بالإدراج ((Insertion Sort** من أبسط خوارزميات الترتيب وأكثرها سهولة في الفهم والتطبيق. تعتمد الخوارزمية على بناء قائمة مرتبة نهائية **عنصرًا بعد عنصر** عن طريق تمريرة متكررة، حيث يتم أخذ كل عنصر من الجزء غير المرتب وإدراجه في موضعه الصحيح داخل الجزء المرتب من القائمة.

مثال عملي:

شرح مفصل لتطبيق الترتيب بالإدراج على القائمة: [8, 3, 9, 2].

آلية عمل الترتيب بالإدراج

1. **الافتراض الأولي:** نفترض أن أول عنصر في القائمة **[8]** هو جزء مرتب بالفعل.
2. **التكرار:** تبدأ الخوارزمية من العنصر الثاني وتستمر حتى نهاية القائمة.
3. **المفتاح (:Key)** يتم اختيار العنصر الحالي كمفتاح.
4. **المقارنة والإزاحة:** يُقارن المفتاح بالعناصر الموجودة في الجزء المرتب (على يساره) بدءًا من اليمين. إذا كان المفتاح **أصغر** من العنصر الذي تتم مقارنته به، يتم إزاحة العنصر الأكبر خطوة واحدة إلى اليمين.
5. **الإدراج:** تتوقف الإزاحة عندما نجد موقعًا يكون فيه المفتاح أكبر من أو يساوي العنصر الذي يسبقه. يتم إدراج المفتاح في هذا الموضع الفارغ.

تطبيق الترتيب بالإدراج على القائمة: [8, 3, 9, 2]

البداية: القائمة غير المرتبة

3	**2**	**1**	**0**
2	9	3	8
			(الجزء المرتب: [8])

التكرار الأول: المفتاح = 3

1. **المفتاح:** نختار الرقم **3** (الموقع 1).
2. **المقارنة:** نقارن **3** بالعنصر الذي يسبقه (8).
   - هل $3 < 8$ ؟ **نعم**.
3. **الإزاحة والإدراج:**
   - نزيح 8 إلى اليمين (الموقع 1).
   - ندرج 3 في الموضع الفارغ (الموقع 0).

3	**2**	**1**	**0**
2	9	**8**	**3**
			(الجزء المرتب: [3, 8])

التكرار الثاني: المفتاح = 9

1. **المفتاح:** نختار الرقم **9** (الموقع 2).
2. **المقارنة:** نقارن **9** بالعناصر التي تسبقه (8، ثم 3).
   - هل $9 < 8$ ؟ **لا**.
3. **الإدراج:** نتوقف. العنصر 9 موجود بالفعل في موضعه الصحيح داخل الجزء المرتب.

3	**2**	**1**	**0**
2	**9**	8	3
			(الجزء المرتب: [3, 8, 9])

التكرار الثالث: المفتاح = 2

1. **المفتاح:** نختار الرقم **2** (الموقع 3).
2. **المقارنة والإزاحة:** نبدأ المقارنة من يمين الجزء المرتب (9) وننتقل إلى اليسار:
   - نقارن **2** بـ 9. هل $2 < 9$ ؟ **نعم**. (نزيح 9).
   - نقارن **2** بـ 8. هل $2 < 8$ ؟ **نعم**. (نزيح 8).
   - نقارن **2** بـ 3. هل $2 < 3$ ؟ **نعم**. (نزيح 3).
   - نصل إلى بداية القائمة.

3. **الإدراج:** ندرج 2 في الموضع 0.

3	**2**	**1**	**0**
9	8	3	**2**
			(الجزء المرتب: [2, 3, 8, 9])

النتيجة النهائية

تم الانتهاء من جميع التكرارات، والقائمة الآن مرتبة بالكامل.

[2, 3, 8, 9]

تحليل الأداء:(Insertion Sort)

- **تعقيد الوقت:**
- **الحالة الأسوأ :(Worst Case)** عندما تكون القائمة مرتبة عكسياً. في هذه الحالة، كل عنصر جديد سيحتاج إلى مقارنته مع كل عناصر الجزء المرتب، مما يؤدي إلى تعقيد .**$O(n^2)$**
- **الحالة الأفضل :(Best Case)** عندما تكون القائمة مرتبة بالفعل. في هذه الحالة، كل عنصر جديد سيحتاج إلى مقارنة واحدة فقط مع آخر عنصر في الجزء المرتب. بما أننا نمر على كل العناصر، يكون التعقيد .**$\Omega(n)$**
- **Theta (Θ):** لا يمكن استخدامه لوصف الخوارزمية بشكل عام.
- **ميزة خاصة :**هذه الخوارزمية فعالة جداً مع **القوائم شبه المرتبة** والقوائم الصغيرة. كما أنها خوارزمية **"متصلة(Online) "**، أي يمكنها ترتيب القائمة أثناء استقبال عناصرها.

2- ترتيب شيل (Shell Sort)

الفكرة الأساسية :لاحظ "دونالد شيل" أن الترتيب بالإضافة يكون بطيئاً لأن العناصر تتحرك خطوة واحدة فقط في كل مرة. فماذا لو سمحنا للعناصر بالقفز لمسافات كبيرة عبر القائمة؟ ترتيب شيل هو نسخة محسنة من الترتيب بالإضافة تسمح بتبادل العناصر البعيدة عن بعضها.

خطوات الخوارزمية:

1. نبدأ بفجوة (gap) كبيرة. حجم الفجوة يمكن حسابه بعدة طرق، والطريقة الكلاسيكية هي .n/2
2. نقوم بترتيب عناصر القائمة باستخدام الترتيب بالإضافة، ولكن بدلاً من مقارنة العناصر المتجاورة، نقارن العناصر التي تفصل بينها هذه "الفجوة". (مثلاً، نقارن العنصر 1 مع العنصر 1+gap، والعنصر 2 مع العنصر 2+gap، وهكذا). هذا يجعل القائمة "شبه مرتبة" بسرعة.

3. نقوم بتقليص الفجوة (مثلاً، نقسمها على 2).
4. نكرر عملية الترتيب بالإضافة مع الفجوة الجديدة الأصغر.
5. نستمر في تقليص الفجوة وتكرار الترتيب حتى تصبح الفجوة تساوي 1.
6. عندما تكون الفجوة 1، فإن الخوارزمية هي تماماً مثل الترتيب بالإضافة العادي. لكن لأن القائمة أصبحت شبه مرتبة إلى حد كبير من الخطوات السابقة، فإن هذه الخطوة الأخيرة تكون سريعة جداً.

مثال عملي :

لتطبيق **ترتيب شيل ((Shell Sort** على القائمة **[66,5,17,93,41,11,54,43]** (حجم n=8)،

متسلسلة الفجوات المستخدمة: [4,1]

المرحلة 1: الترتيب بفجوة h=4

نقسم القائمة إلى أربع قوائم فرعية ((sublistsغير متجاورة، ونرتب كل قائمة باستخدام الترتيب بالإدراج.

القوائم الفرعية (:Sublists)

1. **الفهرس 0:** [...,41,...,66]
2. **الفهرس 1:** [...,11,...,5]
3. **الفهرس 2:** [...,54,...,17]
4. **الفهرس 3:** [...,43,...,93]

الفهرس	0	1	2	3	4	5	6	7
القائمة الأصلية	**66**	**5**	**17**	**93**	**41**	**11**	**54**	**43**
الفجوة 4	*القائمة 1*	*القائمة 2*	*القائمة 3*	*القائمة 4*	*القائمة 1*	*القائمة 2*	*القائمة 3*	*القائمة 4*

ترتيب القوائم الفرعية:

- **القائمة 1:** [41,66]→[66,41]
- **القائمة 2:** [5,11]→[5,11] (مرتبة بالفعل)
- **القائمة 3:** [17,54]→[17,54] (مرتبة بالفعل)
- **القائمة 4:** [43,93]→[93,43]

القائمة بعد المرحلة 1:

الفهرس 0 1 2 3 4 5 6 7
القائمة الوسيطة 41 5 17 43 66 11 54 93

المرحلة 2: الترتيب بفجوة h=1

نستخدم الترتيب بالإدراج العادي على القائمة الناتجة. بما أن العناصر قد تحركت كثيرًا بالفعل، ستكون عملية الترتيب هذه سريعة (أقل تبديلات).

القائمة الحالية: [41,5,17,43,66,11,54,93]

التكرار (المفتاح)	القائمة الوسيطة	التبديلات (المقارنة)
المفتاح 5	[5,41,17,43,66,11,54,93]	5أدخلت قبل 41
المفتاح 17	[5,17,41,43,66,11,54,93]	17أدخلت قبل 41
المفتاح 43	[5,17,41,43,66,11,54,93]	43في مكانها الصحيح
المفتاح 66	[5,17,41,43,66,11,54,93]	66في مكانها الصحيح
المفتاح 11	[5,11,17,41,43,66,54,93]	11أدخلت قبل 66، 43، 41، 17
المفتاح 54	[5,11,17,41,43,54,66,93]	54أدخلت قبل 66
المفتاح 93	[5,11,17,41,43,54,66,93]	93في مكانها الصحيح

القائمة النهائية المرتبة

بعد الانتهاء من جميع المراحل:

[5,11,17,41,43,54,66,93]
تحليل الأداء:(Shell Sort)

- **تعقيد الوقت:**

- تحليل ترتيب شيل معقد جداً ويعتمد بشكل كبير على "تسلسل الفجوات" المستخدم.
- إنه ليس $O(n^2)$. أفضل تسلسلات الفجوات المعروفة تصل بتعقيد الوقت إلى ما يقارب **O(n log n)** أو **O(n^(4/3))**.
- في كل الأحوال، هو أسرع بكثير من خوارزميات $O(n^2)$ البسيطة.
- **تعقيد المساحة**O(1) :، لأنه لا يحتاج إلى ذاكرة إضافية.

الترتيب بالإضافة هو خوارزمية بسيطة وفعالة جداً للقوائم الصغيرة وشبه المرتبة. ترتيب شيل يأخذ هذه الفكرة ويطورها للسماح للعناصر بالقفز لمسافات طويلة، مما يجعله خياراً أسرع بكثير للقوائم متوسطة الحجم، متفوقاً على خوارزميات $O(n^2)$ البسيطة.